Patrones de software

Implementación de patrones v3

Docente: ELIECER MONTERO OJEDA

Ángel David Reyes Moreno

UNIDADES TECNOLÓGICAS DE SANTANDER
Ingenieria de Sistemas
Bucaramanga, 2026

Patrón 1: Singleton

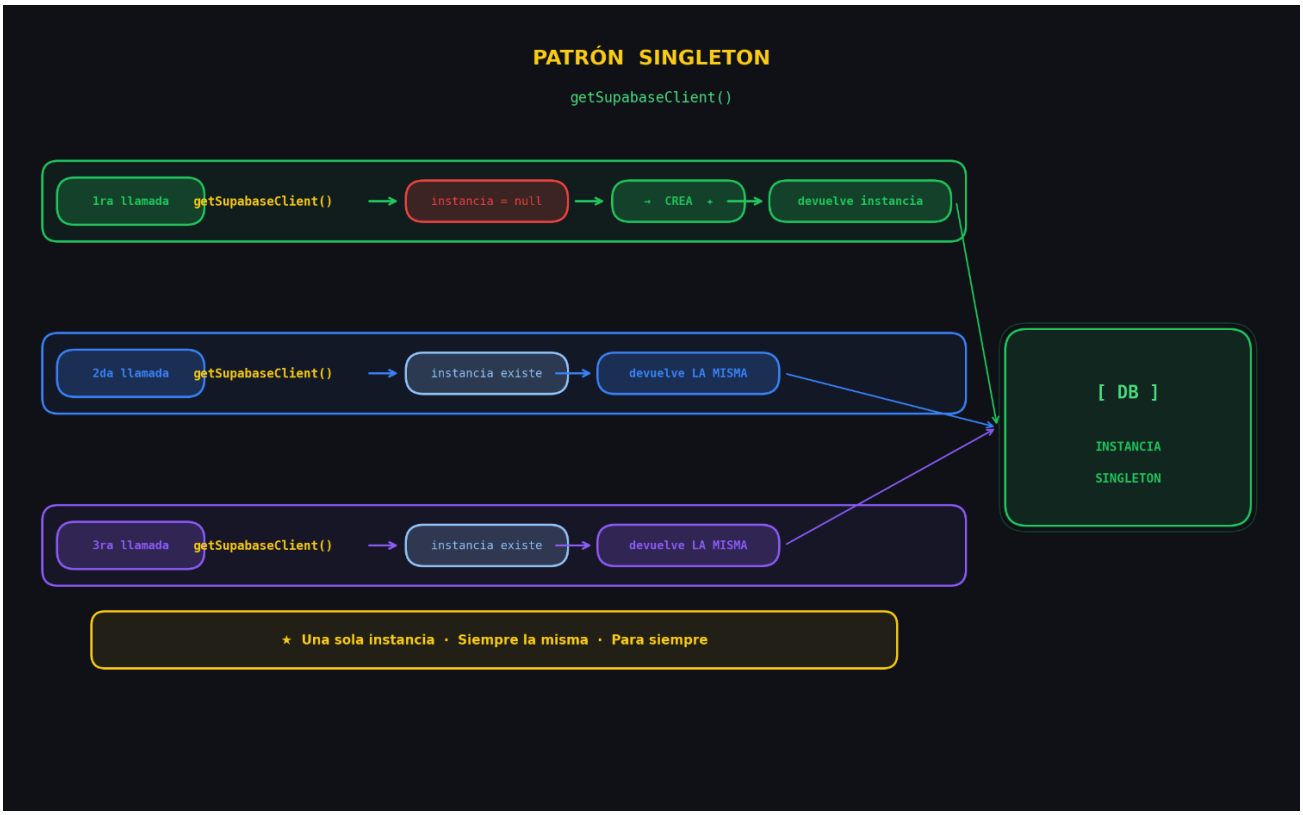
¿Qué problema resuelve? Garantiza que solo exista una única instancia de un objeto durante toda la vida de la aplicación. Útil para configuración y conexiones a bases de datos.

```
export class ConfigManager {
  // 1. Variable estática que guarda LA única instancia
  private static instance: ConfigManager | null = null;

  // 2. Constructor PRIVADO → nadie puede hacer "new ConfigManager()"
  private constructor() {
    this.config = { /* lee las variables de entorno */ }
  }

  // 3. Punto de acceso global → siempre devuelve la misma instancia
  public static getInstance(): ConfigManager {
    if (!ConfigManager.instance) {
      ConfigManager.instance = new ConfigManager(); // solo se crea UNA vez
    }
    return ConfigManager.instance; // las demás veces, devuelve la existente
  }
}
```

¿Dónde se usa? El cliente de Supabase lo llama al iniciarse:

```
let supabaseInstance: SupabaseClient | null = null;

export function getSupabaseClient(): SupabaseClient {
  if (supabaseInstance) return supabaseInstance; // ya existe → devuelve la misma

  const config = ConfigManager.getInstance(); // ← usa el otro Singleton
  supabaseInstance = createClient(config.get("supabaseUrl"), ...);
  return supabaseInstance;
}
```

Diagrama Uml


PATRÓN SINGLETON
getSupabaseClient()
1ra llamada
getSupabaseClient()
instancia = null
→ CREA ✦
devuelve instancia
2da llamada
getSupabaseClient()
instancia existe
devuelve LA MISMA
3ra llamada
getSupabaseClient()
instancia existe
devuelve LA MISMA
[DB]
INSTANCIA
SINGLETON
★ Una sola instancia · Siempre la misma · Para siempre

Patrón 2: Factory Method

¿Qué problema resuelve? Delegar la **creación de objetos** a subclases, sin que el código cliente sepa qué clase concreta se está creando. Perfecto para múltiples tipos de dispositivos IoT.

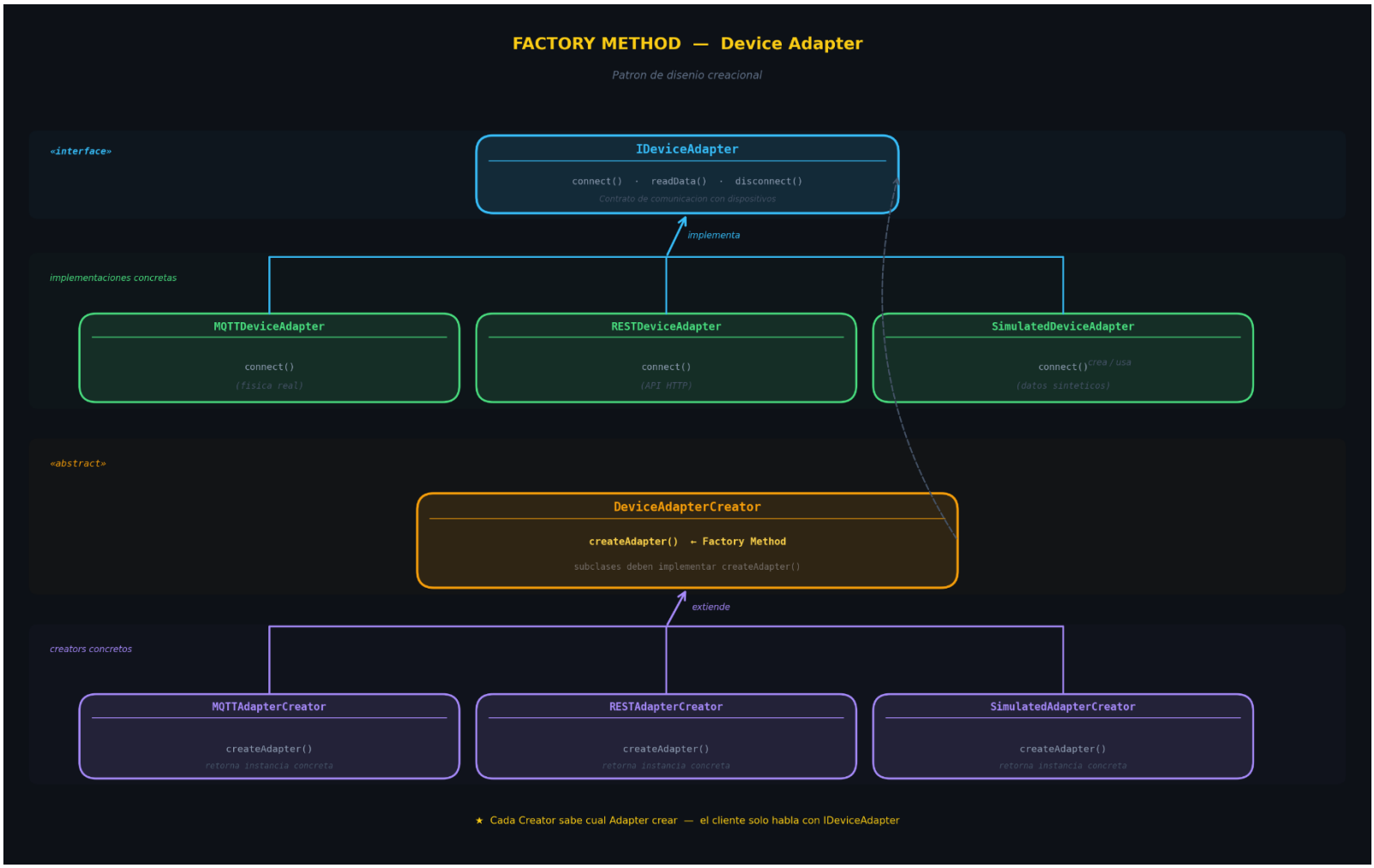
```
// Creator abstracto define el "molde"
abstract class DeviceAdapterCreator {
  abstract createAdapter(id: string, type: DeviceType): IDeviceAdapter; // ← Factory Method
}

// Cada subclase decide QUÉ adaptador concreto crear
class SimulatedAdapterCreator extends DeviceAdapterCreator {
  createAdapter(id: string, type: DeviceType): IDeviceAdapter {
    return new SimulatedDeviceAdapter(id, type); // concreta
  }
}

// Punto de entrada público — elige el Creator correcto
export class DeviceAdapterFactory {
  static async create(params): Promise<IDeviceAdapter> {
    switch (params.protocol) {
      case "MQTT":      return new MQTTAdapterCreator().getAndConnect(...);
      case "REST":      return new RESTAdapterCreator().getAndConnect(...);
      case "SIMULATED": return new SimulatedAdapterCreator().getAndConnect(...);
    }
  }
}
```

¿Dónde se usa en la UI?

La UI solo llama a la fábrica. De momento se esta instanciando un protocolo de simulación si en un futuro se implementa un protocolo en tiempo reaL solo hay que crear ese protocolo ejemplo: ZIGBEE, solo ZigbeeDeviceAdapter y ZigbeeAdapterCreator — el código de la UI no cambia para nada seguirá llamando a la fabrica .

```
// La página NO sabe qué adaptador concreto se crea
const adapter = await DeviceAdapterFactory.create({
  deviceId: "dev-001",
  deviceType: "SOLAR_PANEL",
  protocol: "SIMULATED",  // ← aquí solo cambia el protocolo
});
const reading = await adapter.readData(); // misma interfaz siempre
```

Diagrama Uml


FACTORY METHOD — Device Adapter
Patron de disenio creacional
«interface»
IDeviceAdapter
connect() · readData() · disconnect()
Contrato de comunicacion con dispositivos
implementa
implementaciones concretas
MQTTDeviceAdapter
connect()
(fisica real)
RESTDeviceAdapter
connect()
(API HTTP)
SimulatedDeviceAdapter
connect()
crea / usa
(datos sinteticos)
«abstract»
DeviceAdapterCreator
createAdapter() ← Factory Method
subclases deben implementar createAdapter()
extiende
creators concretos
MQTTAdapterCreator
createAdapter()
retorna instancia concreta
RESTAdapterCreator
createAdapter()
retorna instancia concreta
SimulatedAdapterCreator
createAdapter()
retorna instancia concreta
★ Cada Creator sabe cual Adapter crear — el cliente solo habla con IDeviceAdapter

Patrón 3: Abstract Factory

¿Qué problema resuelve?

En la plataforma tenemos **4 fuentes de energía** (Solar, Eólica, Batería, Red). Cada una necesita tres componentes que **deben ser coherentes entre sí**:

- Un lector de producción
- Un calculador de precios
- Un modelo de predicción

Sin el patrón, la UI tendría que saber qué clase concreta instanciar en cada caso, generando código espagueti

¿Dónde se usa?

Se define las tres faces del producto

```
interface IEnergyProducer   { getCurrentReading(): Promise<EnergyReading>; }
interface IPriceEstimator   { getCurrentQuote(): Promise<PriceQuote>; }
interface IForecastModel    { getDailyForecast(): Promise<DailyForecast>; }
```

Se define la Abstract Factory (el contrato de la fábrica)

```
interface IEnergySourceFactory {
  createProducer(): IEnergyProducer;       // ← crea producto A
  createPriceEstimator(): IPriceEstimator;  // ← crea producto B
  createForecastModel(): IForecastModel;    // ← crea producto C
}
```

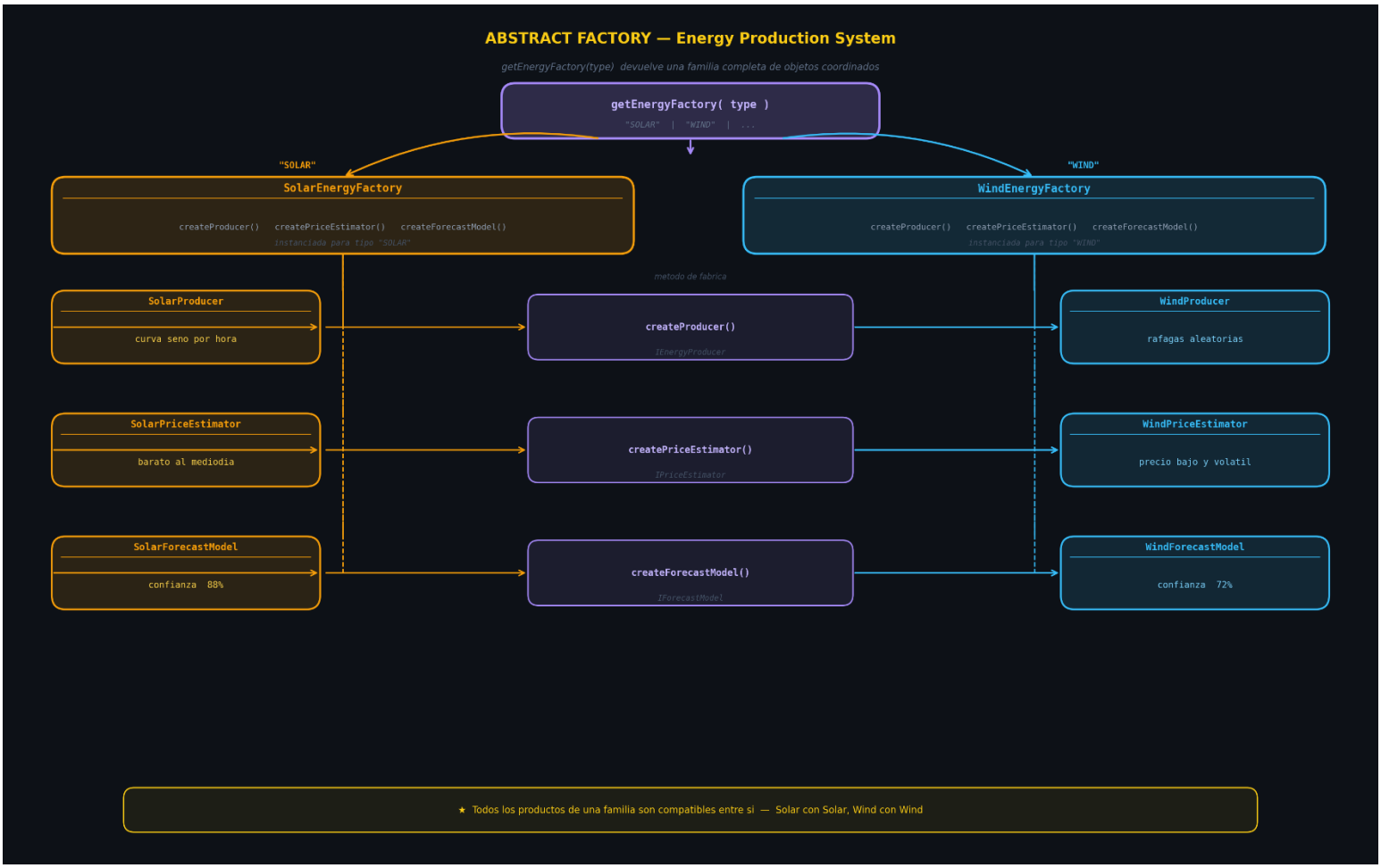
Se implementan las 4 fábricas concretas, una por familia

```
class SolarEnergyFactory implements IEnergySourceFactory {
  createProducer()       { return new SolarProducer(); }      // curva seno solar
  createPriceEstimator() { return new SolarPriceEstimator(); } // más barato al mediodía
  createForecastModel()  { return new SolarForecastModel(); }  // confianza 88%
}

class WindEnergyFactory ... // variación tipo ráfaga, confianza 72%
class BatteryEnergyFactory ... // carga/descarga, confianza 95%
class GridEnergyFactory ... // ilimitada, la más cara, confianza 99%
```

Por ultimo el selector de las cuatro clases de fabrica

```
export function getEnergyFactory(source: EnergySource): IEnergySourceFactory {
  switch (source) {
    case "SOLAR":   return new SolarEnergyFactory();
    case "WIND":    return new WindEnergyFactory();
    case "BATTERY": return new BatteryEnergyFactory();
    case "GRID":    return new GridEnergyFactory();
  }
}
```

Diagrama Uml



Patrón 4: Builder

¿Qué problema resuelve?

Una TradeOrder tiene muchos campos opcionales. Sin Builder, el constructor se convierte en un desastre:

```
// ❌ Sin Builder — imposible saber qué significa cada argumento
new TradeOrder("SELL", 12.5, 0.124, "SOLAR", "FIXED", "URGENT", 30, true, false, "Excedente")
```

Con Builder se construye la orden **paso a paso**, solo con lo que se necesita :

```
new TradeOrderBuilder()
  .ofType("SELL")
  .withAmount(12.5)
  .atPrice(0.124)
  .fromSource("SOLAR")
  .withPriority("URGENT")
  .build()
```

¿Dónde se usa?

El patrón tiene **3 participantes** hasta el momento en el proyecto

1. El Producto — TradeOrder (interfaz)

El objeto complejo que se construye al final:

```
interface TradeOrder {
  id: string;          type: OrderType;       amountKwh: number;
  pricePerKwh: number; energySource: EnergySource;
  pricingMode: PricingMode;  priority: OrderPriority;
  expiresAt: Date | null;    note: string | null;
  conditions: TradeOrderConditions;
  maxSlippagePercent: number;  totalValueUsd: number;
  status: OrderStatus;
}
```

2. El Builder — TradeOrderBuilder

Acumula el estado internamente y expone métodos encadenables:

```
class TradeOrderBuilder {
  // campos privados con valores por defecto seguros
  private _type: OrderType | null = null;
  private _amountKwh: number | null = null;
  private _priority: OrderPriority = "NORMAL";  // default
  private _conditions: TradeOrderConditions = {};
  // ...

  ofType(type)            { this._type = type;       return this; }
  withAmount(kwh)         { this._amountKwh = kwh;   return this; }
  atPrice(usd)            { this._pricePerKwh = usd;  return this; }
  fromSource(src)         { this._energySource = src; return this; }
  expiresInMinutes(min)   { /* calcula Date */        return this; }
  requireGreenCertified() { this._conditions.requireGreenCertified = true; return this; }

  build(): TradeOrder {
    // Valida campos requeridos
    if (!this._type) throw new TradeOrderBuildError("El tipo es requerido");
    if (this._pricePerKwh <= 0) throw new TradeOrderBuildError("El precio debe ser > 0");
    // Calcula campos derivados y retorna el producto final
    return { id: `ORD-...`, totalValueUsd: kwh * price, status: "DRAFT", ... };
  }

  reset() { /* limpia todo para reutilizar */ return this; }
}
```

3. El Director — TradeOrderDirector

Encapsula **recetas predefinidas** usando el Builder. Sabe qué pasos llamar y en qué orden:

```
class TradeOrderDirector {
  constructor(private builder: TradeOrderBuilder) {}

  buildUrgentSolarSell(kwh, price): TradeOrder {
    return this.builder.reset()
      .ofType("SELL").withAmount(kwh).atPrice(price)
      .fromSource("SOLAR").withPricingMode("DYNAMIC")
      .withPriority("URGENT").expiresInMinutes(30)
      .withMaxSlippage(5).allowPartialFill()
      .build();
  }

  buildScheduledGreenBuy(kwh, maxPrice): TradeOrder { ... }
  buildPeakBatteryDischarge(kwh, price): TradeOrder { ... }
}
```

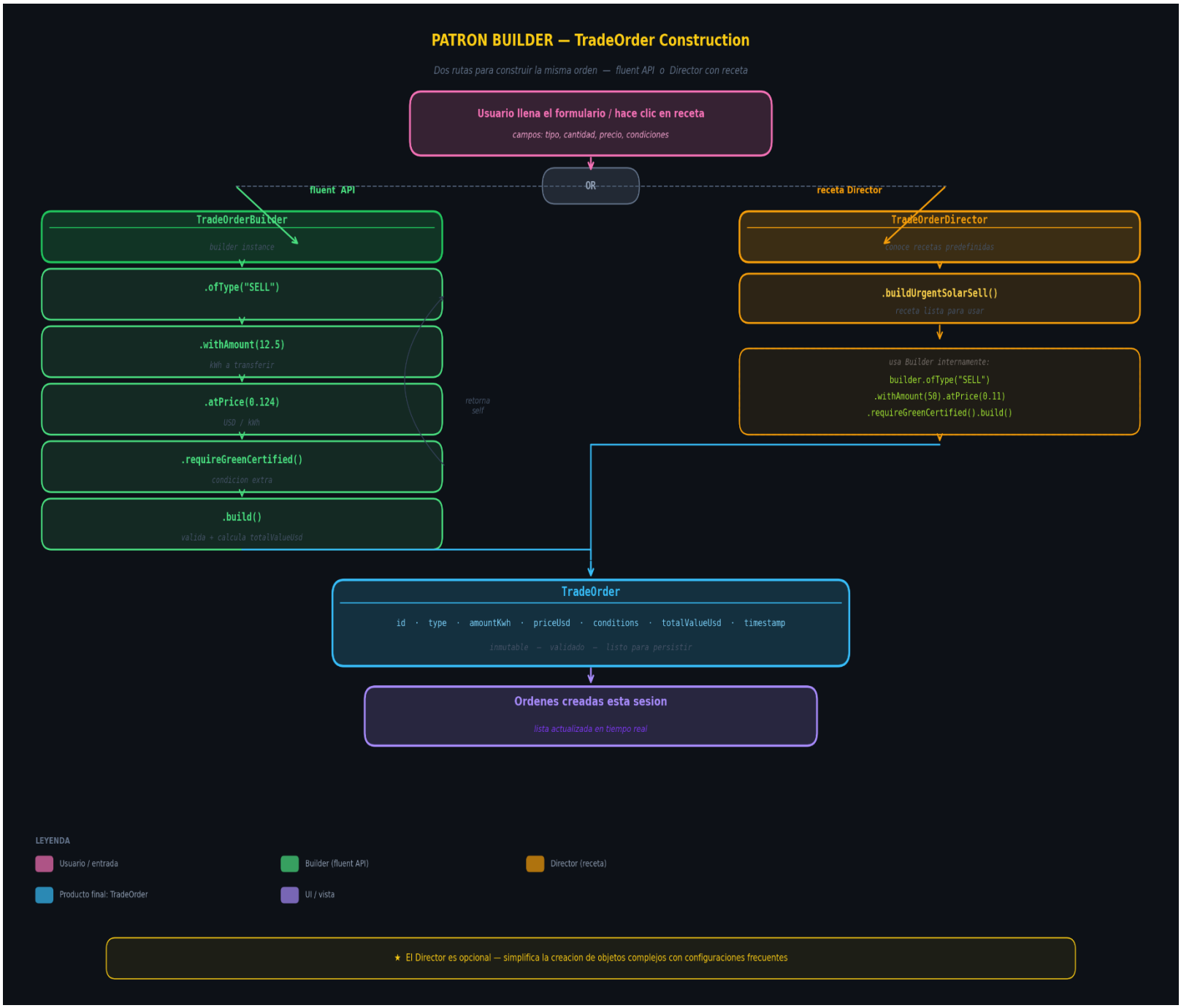



Diagrama Uml